

Лабораторная работа №1

Простые модели компьютерной сети

Шубнякова Дарья НКНбд-01-22

1. Вводная часть

2. Основная часть

3. Результаты

1. Вводная часть

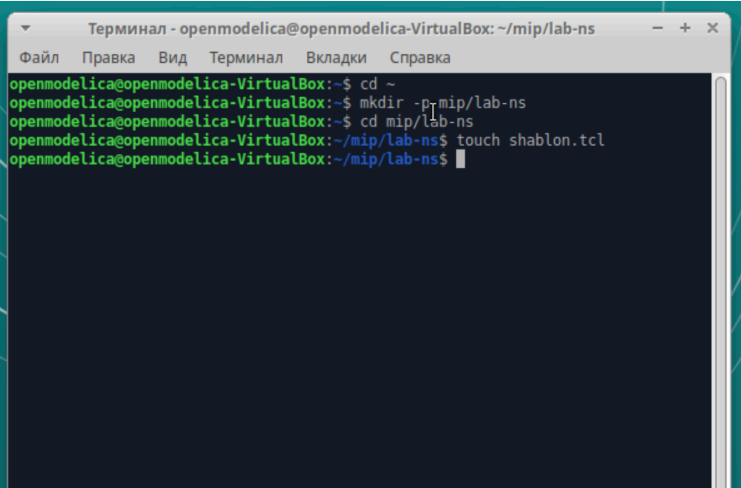
1.1 Цели и задачи

Приобретение навыков моделирования сетей передачи данных с помощью средства имитационного моделирования NS-2, а также анализ полученных результатов моделирования.

2. Основная часть

2.1 Выполнение лабораторной работы

Создаем директорию `mip` для работы с NS и создаем шаблон для дальнейшего выполнения лабораторной.



```
Терминал - openmodelica@openmodelica-VirtualBox: ~/mip/lab-ns
Файл  Правка  Вид  Терминал  Вкладки  Справка
openmodelica@openmodelica-VirtualBox:~$ cd ~
openmodelica@openmodelica-VirtualBox:~$ mkdir -p mip/lab-ns
openmodelica@openmodelica-VirtualBox:~$ cd mip/lab-ns
openmodelica@openmodelica-VirtualBox:~/mip/lab-ns$ touch shablon.tcl
openmodelica@openmodelica-VirtualBox:~/mip/lab-ns$
```

2.2 Берем шаблон из задания и у нас запускается пустой симулятор.

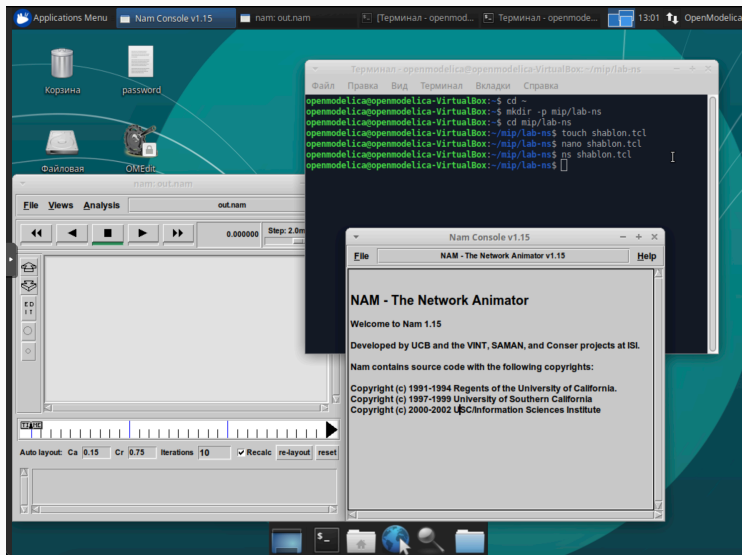
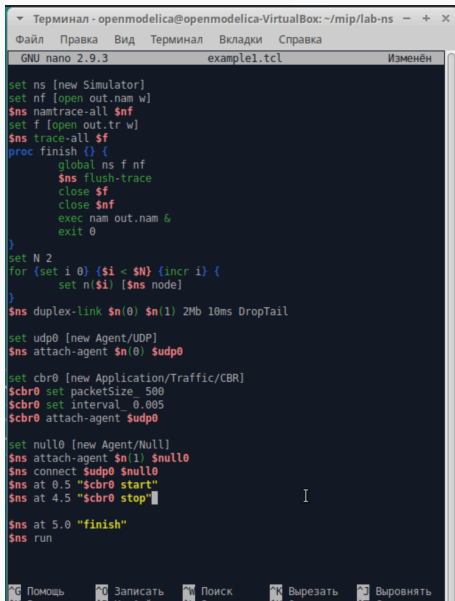


Рисунок 2: Пустой симулятор

2.3 Работаем с первым примером из задания, смотрим, что будет происходить.



```
Терминал - openmodelica@openmodelica-VirtualBox: ~/mip/lab-ns
Файл  Правка  Вид  Терминал  Вкладки  Справка
GNU nano 2.9.3  example1.tcl  Изменён

set ns [new Simulator]
set nf [open out.nam w]
$ns namtrace-all $nf
set f [open out.tr w]
$ns trace-all $f
proc finish () {
    global ns f nf
    $ns flush-trace
    close $f
    close $nf
    exec nam out.nam &
    exit 0
}
set N 2
for {set i 0} {$i < $N} {incr i} {
    set n($i) [$ns node]
}
$ns duplex-link $n(0) $n(1) 2Mb 10ms DropTail

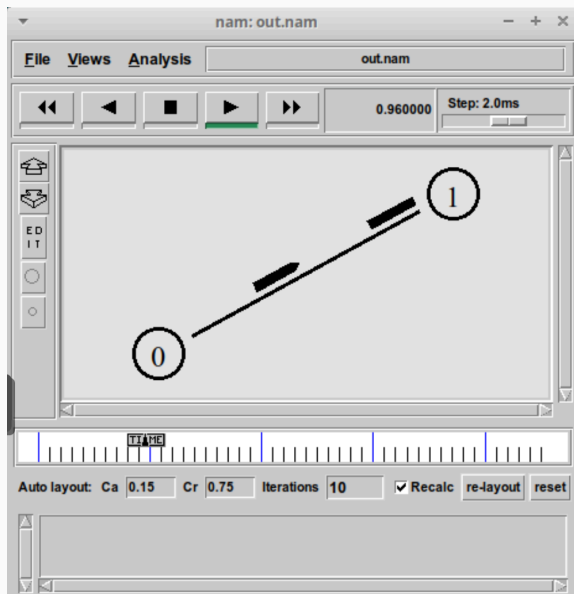
set udp0 [new Agent/UDP]
$ns attach-agent $n(0) $udp0

set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0

set null0 [new Agent/Null]
$ns attach-agent $n(1) $null0
$ns connect $udp0 $null0
$ns at 0.5 "$cbr0 start"
$ns at 4.5 "$cbr0 stop"

$ns at 5.0 "finish"
$ns run
```


2.4 У нас появилась сеть из двух узлов и идущие пакеты из 0 в 1.



2.5 Берем код для второго примера и вставляем его, параллельно читая все комментарии к тому, что делаем.

```
Терминал - openmodelica@openmodelica-VirtualBox: ~/mip/lab-ns
Файл  Правка  Вид  Терминал  Вкладки  Справка
GNU nano 2.9.3  example2.tcl  Изменён

set ns [new Simulator]
set nf [open out.nam w]
$ns namtrace-all $nf
set f [open out.tr w]
$ns trace-all $f
proc finish {} {
    global ns f nf
    $ns flush-trace
    close $f
    close $nf
    exec nam out.nam &
    exit 0
}

set N 4
for {set i 0} {$i < $N} {incr i} {
    set n($i) [$ns node]

    $ns duplex-link $n(0) $n(2) 2Mb 10ms DropTail
    $ns duplex-link $n(1) $n(2) 2Mb 10ms DropTail
    $ns duplex-link $n(3) $n(2) 2Mb 10ms DropTail
    $ns duplex-link-op $n(0) $n(2) orient right-down
    $ns duplex-link-op $n(1) $n(2) orient right-up
    $ns duplex-link-op $n(2) $n(3) orient right

    # создание агента UDP и присоединение его к узлу n(0)
    set udp0 [new Agent/UDP]
    $ns attach-agent $n(0) $udp0
    # создание источника CBR-трафика
    # и присоединение его к агенту udp0
    set cbr0 [new Application/Traffic/CBR]
    $cbr0 set packetSize_ 500
    $cbr0 set interval_ 0.005
    $cbr0 attach-agent $udp0
    # создание агента TCP и присоединение его к узлу n(1)
    set tcp1 [new Agent/TCP]
```

```
Терминал - openmodelica@openmodelica-VirtualBox: ~/mip/lab-ns - + x
Файл Правка Вид Терминал Вкладки Справка
GNU nano 2.9.3 example2.tcl Изменён

$ns duplex-link $n(1) $n(2) 2Mb 10ms DropTail
$ns duplex-link $n(3) $n(2) 2Mb 10ms DropTail
$ns duplex-link-op $n(0) $n(2) orient right-down
$ns duplex-link-op $n(1) $n(2) orient right-up
$ns duplex-link-op $n(2) $n(3) orient right

# создание агента UDP и присоединение его к узлу n(0)
set udp0 [new Agent/UDP]
$ns attach-agent $n(0) $udp0
# создание источника CBR-трафика
# и присоединение его к агенту udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize 500
$cbr0 set interval 0.005
$cbr0 attach-agent $udp0

# создание агента TCP и присоединение его к узлу n(1)
set tcp1 [new Agent/TCP]
$ns attach-agent $n(1) $tcp1
# создание приложения FTP
# и присоединение его к агенту tcp1
set ftp [new Application/FTP]
$ftp attach-agent $tcp1
# создание агента-получателя для udp0
set null0 [new Agent/Null]
$ns attach-agent $n(3) $null0
# создание агента-получателя для tcp1
set sink1 [new Agent/TCPSink]
$ns attach-agent $n(3) $sink1
# Соединим агенты udp0 и tcp1 и их получателей:
$ns connect $udp0 $null0
$ns connect $tcp1 $sink1
# Зададим описание цвета каждого потока:
$ns color 1 Blue
$ns color 2 Red
$udp0 set class_ 1
$tcp1 set class_ 2
# Отслеживание событий в очереди:

^C Помощь ^O Записать ^M Поиск ^K Вырезать ^J Выворачивать
^X Выход ^R ЧитФайл ^N Замена ^U Отмен. выре ^T Словарь
```

2.6

```

Терминал - openmodelica@openmodelica-VirtualBox: ~/mip/lab-ns
Файл Правка Вид Терминал Вкладки Справка
GNU nano 2.9.3 example2.tcl Изменён

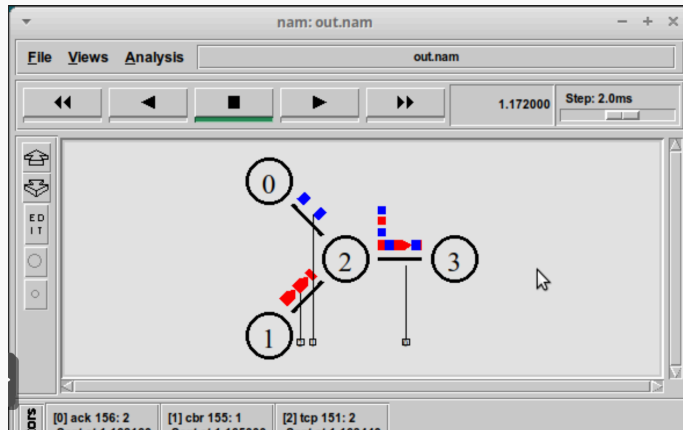
# создание приложения FTP
# и присоединение его к агенту tcp1
set ftp [new Application/FTP]
$ftp attach-agent $tcp1
# создание агента-получателя для udp0
set null0 [new Agent/Null]
$ns attach-agent $n(3) $null0
# создание агента-получателя для tcp1
set sink1 [new Agent/TCPSink]
$ns attach-agent $n(3) $sink1
# Соединим агенты udp0 и tcp1 и их получателей:
$ns connect $udp0 $null0
$ns connect $tcp1 $sink1
# Зададим описание цвета каждого потока:
$ns color 1 Blue
$ns color 2 Red
$udp0 set class 1
$tcp1 set class 2
# Отслеживание событий в очереди:
$ns duplex-link-op $n(2) $n(3) queuePos 0.5
# Наложение ограничения на размер очереди:
$ns queue-limit $n(2) $n(3) 20
# Добавление ат-событий:
$ns at 0.5 "$cbr0 start"
$ns at 1.0 "$ftp start"
$ns at 4.0 "$ftp stop"
$ns at 4.5 "$cbr0 stop"

$ns at 5.0 "finish"
$ns run

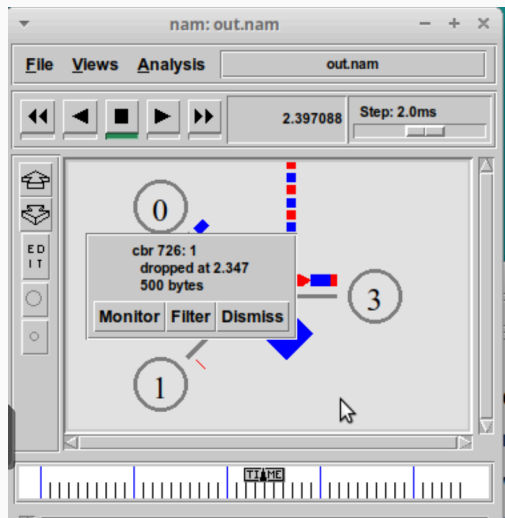
^G Помощь ^O Записать ^W Поиск ^K Вырезать ^J Выровнять ^C ТекПозиц
^X Выход ^R ЧитФайл ^M Замена ^U Отмен. выр ^T Словарь ^_ К строке

```

2.8 При запуске скрипта можно заметить, что по соединениям между узлами $n(0)$ – $n(2)$ и $n(1)$ – $n(2)$ к узлу $n(2)$ передаётся данных больше, чем способно передаваться по соединению от узла $n(2)$ к узлу $n(3)$. Действительно, мы передаём 200 пакетов в секунду от каждого источника данных в узлах $n(0)$ и $n(1)$, а каждый пакет имеет размер 500 байт. Таким образом, полоса каждого соединения 0,8 Mb, а суммарная — 1,6 Mb.



2.9 Но соединение n(2)–n(3) имеет полосу лишь 1 Mb. Следовательно, часть пакетов должна теряться. В окне аниматора можно видеть пакеты в очереди, а также те пакеты, которые отбрасываются при переполнении.



2.10 Далее мы строим третий пример с кольцевой топологией сети. Вводим код, данный в задании.

```
Терминал - openmodelica@openmodelica-VirtualBox: ~/mip/lab-ns
Файл  Правка  Вид  Терминал  Вкладки  Справка
GNU nano 2.9.3                                example3.tcl

set ns [new Simulator]
$ns rtproto DV
set nf [open out.nam w]
$ns namtrace-all $nf
set f [open out.tr w]
$ns trace-all $f
proc finish {} {
    global ns f nf
    $ns flush-trace
    close $f
    close $nf
    exec nam out.nam &
    exit 0
}

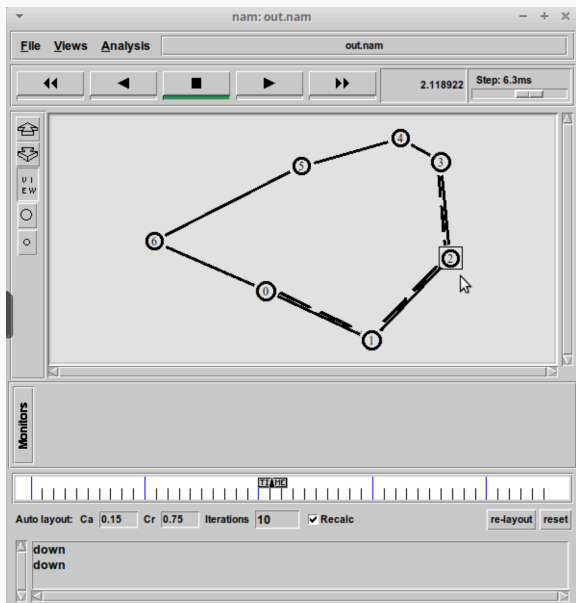
set N 7
for {set i 0} {$i < $N} {incr i} {
    set n($i) [$ns node]
}

for {set i 0} {$i < $N} {incr i} {
    $ns duplex-link $n($i) $n([expr ($i+1)%$N]) 1Mb 10ms DropTail
}

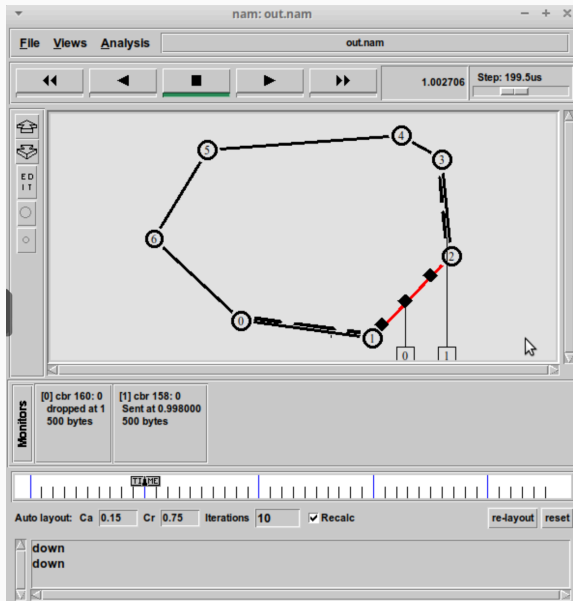
set udp0 [new Agent/UDP]
$ns attach-agent $n(0) $udp0
set cbr0 [new Agent/CBR]
$ns attach-agent $n(0) $cbr0
$cbr0 set packetSize 500
$cbr0 set interval 0.005
set null0 [new Agent/Null]
$ns attach-agent $n(3) $null0
$ns connect $cbr0 $null0

$ns at 0.5 "$cbr0 start"
$ns rtmodel-at 1.0 down $n(1) $n(2)
$ns rtmodel-at 2.0 up $n(1) $n(2)
$ns at 4.5 "$cbr0 stop"
$ns at 5.0 "finish"
```

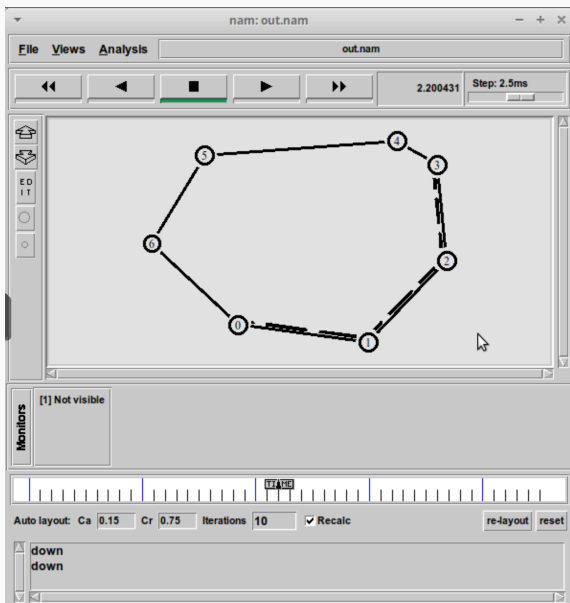
2.11 Видим, как у нас идут пакеты из 0 в 3.



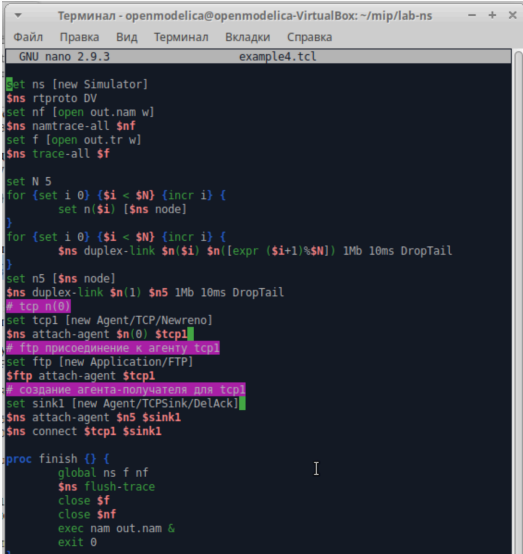
2.12 Во время разрыва соединения у нас происходит утечка пакетов.



2.14 После восстановления сети пакеты снова движутся по короткому пути.



2.15 Пишем код для упражнения. Его можно найти в моем репозитории на GitHub [dishubnyakova/simmod](https://github.com/dishubnyakova/simmod) в документах к первой лабораторной работе.



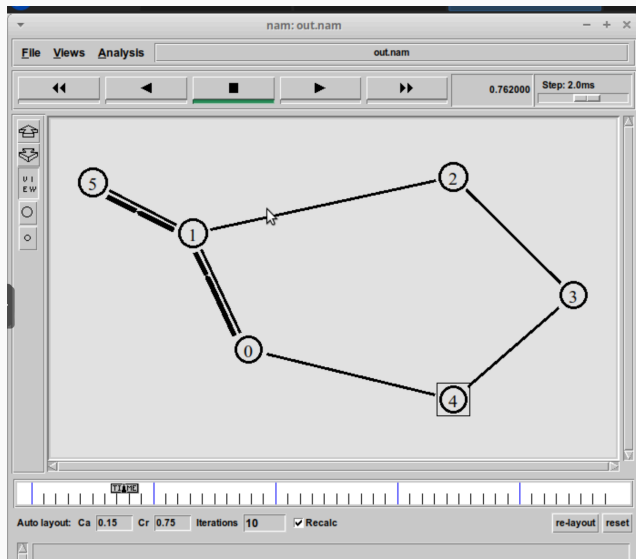
```
Терминал - openmodelica@openmodelica-VirtualBox: ~/mip/lab-ns
Файл  Правка  Вид  Терминал  Вкладки  Справка
GNU nano 2.9.3 example4.tcl

set ns [new Simulator]
$ns rtproto DV
set nf [open out.nam w]
$ns namtrace-all $nf
set f [open out.tr w]
$ns trace-all $f

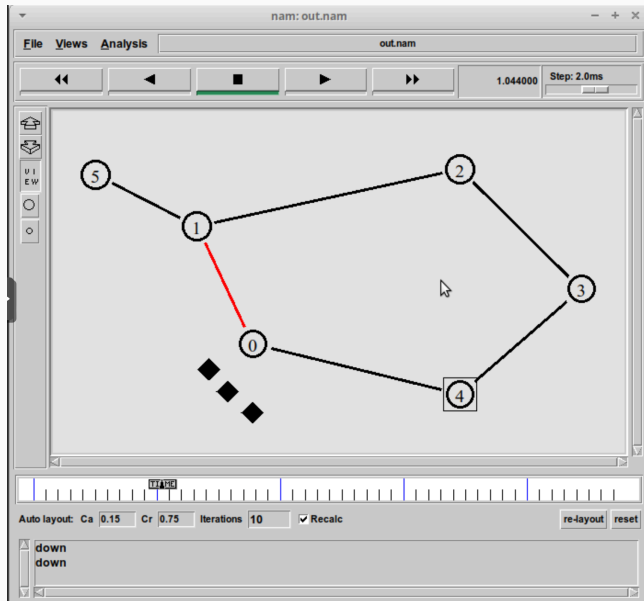
set N 5
for {set i 0} {$i < $N} {incr i} {
    set n($i) [$ns node]
}
for {set i 0} {$i < $N} {incr i} {
    $ns duplex-link $n($i) $n([expr ($i+1)%$N]) 1Mb 10ms DropTail
}
set n5 [$ns node]
$ns duplex-link $n(1) $n5 1Mb 10ms DropTail
# tcp n(0)
set tcp1 [new Agent/TCP/Newreno]
$ns attach-agent $n(0) $tcp1
# ftp присоединение к агенту tcp1
set ftp [new Application/FTP]
$ftp attach-agent $tcp1
# создание агента-получателя для tcp1
set sink1 [new Agent/TCPSink/DelAck]
$ns attach-agent $n5 $sink1
$ns connect $tcp1 $sink1

proc finish {} {
    global ns f nf
    $ns flush-trace
    close $f
    close $nf
    exec nam out.nam &
    exit 0
}
```

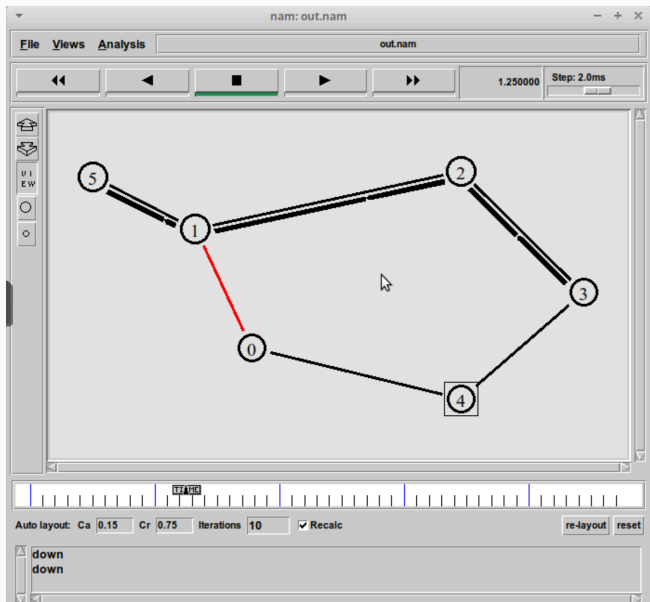
2.16 Тут все как с примером с кольцевой топологией сети кроме того, что нам пришлось подключить узел 5. Видим, как у нас идут пакеты.



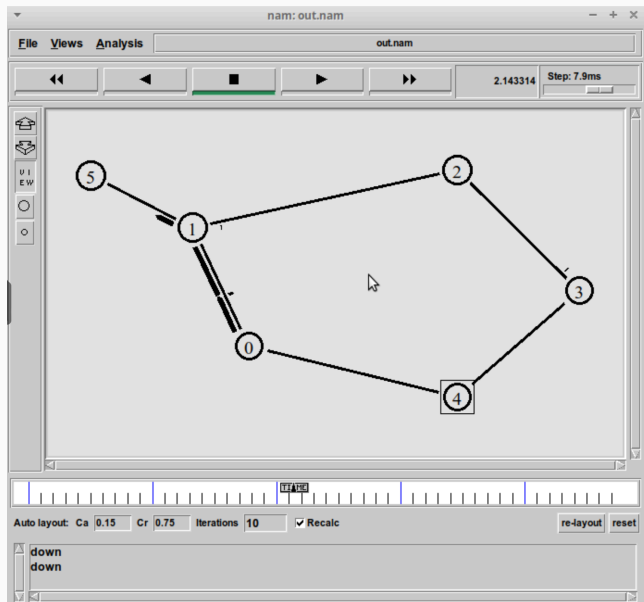
2.17 Потом сеть разрывается, мы теряем пакеты.



2.18 Они находят себе новый путь.



2.19 А после восстановления сети, восстанавливается и передача пакетов.



3. Результаты

3. Результаты

Мы научились делать три вида моделей, из которых потом создали свою собственную из упражнения, где:

- Передача данных осуществляется от узла $n(0)$ до узла $n(5)$ по кратчайшему пути в течение 5 секунд модельного времени;
- Передача данных должна идти по протоколу TCP (тип Newreno), на принимающей стороне используется TCPSink-объект типа DelAck; поверх TCP работает протокол FTP с 0,5 до 4,5 секунд модельного времени;
- С 1 по 2 секунду модельного времени происходит разрыв соединения между узлами $n(0)$ и $n(1)$;
- При разрыве соединения маршрут передачи данных должен измениться на резервный, после восстановления соединения пакеты снова должны пойти по кратчайшему пути.